

```
"""
```

```
WebKom Telegram Bot
```

```
=====
```

```
Збирає інформацію про проект клієнта і пересилає заявку власнику.
```

```
Підтримує 4 мови: CS / EN / RU / UK.
```

```
Ніколи не називає ціну – лише фіксує що клієнт хоче.
```

```
"""
```

```
import os
```

```
import logging
```

```
from datetime import datetime
```

```
from telegram import Update, ReplyKeyboardMarkup, ReplyKeyboardRemove, InlineKeyb
```

```
from telegram.ext import (
```

```
    Application,
```

```
    CommandHandler,
```

```
    MessageHandler,
```

```
    CallbackQueryHandler,
```

```
    ConversationHandler,
```

```
    ContextTypes,
```

```
    filters,
```

```
)
```

```
from texts import T
```

```
# ===== НАЛАШТУВАННЯ =====
```

```
BOT_TOKEN = os.getenv("BOT_TOKEN", "PASTE_YOUR_TOKEN_HERE")
```

```
OWNER_CHAT_ID = int(os.getenv("OWNER_CHAT_ID", "0")) # ваш Telegram chat_id
```

```
logging.basicConfig(
```

```
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
```

```
    level=logging.INFO,
```

```
)
```

```
logger = logging.getLogger(__name__)
```

```
# ===== СТАНИ ДІАЛОГУ =====
```

```
(
```

```
    LANG,
```

```
    NAME,
```

```
    SITE_TYPE,
```

```
    GOAL,
```

```
    PAGES,
```

```
    DESIGN,
```

```
    CONTENT,
```

```
    EXAMPLES,
```

```

    DEADLINE,
    CONTACT,
    NOTES,
    CONFIRM,
) = range(12)

# ===== ДОПОМІЖНІ ФУНКЦІЇ =====
def t(context: ContextTypes.DEFAULT_TYPE, key: str) -> str:
    """Бере переклад за ключем у мові користувача."""
    lang = context.user_data.get("lang", "cs")
    return T.get(lang, T["cs"]).get(key, key)

def kb(rows, one_time=True):
    """Створює ReplyKeyboard з рядків (кожен рядок – список кнопок)."""
    return ReplyKeyboardMarkup(rows, resize_keyboard=True, one_time_keyboard=one_

# ===== ОБРОБНИКИ =====
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    """Старт – вибір мови."""
    context.user_data.clear()
    keyboard = [
        [InlineKeyboardButton("🇸🇰 Čeština", callback_data="lang_cs"),
         InlineKeyboardButton("🇬🇧 English", callback_data="lang_en")],
        [InlineKeyboardButton("🇷🇺 Русский", callback_data="lang_ru"),
         InlineKeyboardButton("🇺🇦 Українська", callback_data="lang_uk")],
    ]
    await update.message.reply_text(
        "👋 Welcome to WebKom!\nVítejte! • Добро пожаловать! • Вітаємо!\n\n"
        "Please choose your language / Vyberte jazyk:",
        reply_markup=InlineKeyboardMarkup(keyboard),
    )
    return LANG

async def set_language(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    """Зберігає вибрану мову і запитує ім'я."""
    query = update.callback_query
    await query.answer()
    lang = query.data.split("_")[1]
    context.user_data["lang"] = lang

    await query.edit_message_text(t(context, "welcome"))
    await query.message.reply_text(t(context, "ask_name"), reply_markup=ReplyKeyb
    return NAME

```

```

async def get_name(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    context.user_data["name"] = update.message.text.strip()
    rows = [
        [t(context, "site_corp"), t(context, "site_landing")],
        [t(context, "site_eshop"), t(context, "site_other")],
    ]
    await update.message.reply_text(
        t(context, "ask_site_type").format(name=context.user_data["name"]),
        reply_markup=kb(rows),
    )
    return SITE_TYPE

```

```

async def get_site_type(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    context.user_data["site_type"] = update.message.text.strip()
    await update.message.reply_text(
        t(context, "ask_goal"),
        reply_markup=ReplyKeyboardRemove(),
    )
    return GOAL

```

```

async def get_goal(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    context.user_data["goal"] = update.message.text.strip()
    rows = [
        [t(context, "pages_1"), t(context, "pages_2_5")],
        [t(context, "pages_5_10"), t(context, "pages_10_plus")],
        [t(context, "pages_unknown")],
    ]
    await update.message.reply_text(t(context, "ask_pages"), reply_markup=kb(rows))
    return PAGES

```

```

async def get_pages(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    context.user_data["pages"] = update.message.text.strip()
    rows = [
        [t(context, "design_have"), t(context, "design_need")],
        [t(context, "design_inspiration")],
    ]
    await update.message.reply_text(t(context, "ask_design"), reply_markup=kb(rows))
    return DESIGN

```

```

async def get_design(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    context.user_data["design"] = update.message.text.strip()

```

```

rows = [
    [t(context, "content_have"), t(context, "content_partial")],
    [t(context, "content_need")],
]
await update.message.reply_text(t(context, "ask_content"), reply_markup=kb(ro
return CONTENT

async def get_content(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    context.user_data["content"] = update.message.text.strip()
    await update.message.reply_text(
        t(context, "ask_examples"),
        reply_markup=ReplyKeyboardMarkup(
            [[t(context, "skip")]], resize_keyboard=True, one_time_keyboard=True
        ),
    )
    return EXAMPLES

async def get_examples(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    txt = update.message.text.strip()
    context.user_data["examples"] = "" if txt == t(context, "skip") else txt
    rows = [
        [t(context, "deadline_asap"), t(context, "deadline_month")],
        [t(context, "deadline_2_3"), t(context, "deadline_flex")],
    ]
    await update.message.reply_text(t(context, "ask_deadline"), reply_markup=kb(r
    return DEADLINE

async def get_deadline(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    context.user_data["deadline"] = update.message.text.strip()
    await update.message.reply_text(
        t(context, "ask_contact"),
        reply_markup=ReplyKeyboardRemove(),
    )
    return CONTACT

async def get_contact(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    context.user_data["contact"] = update.message.text.strip()
    await update.message.reply_text(
        t(context, "ask_notes"),
        reply_markup=ReplyKeyboardMarkup(
            [[t(context, "skip")]], resize_keyboard=True, one_time_keyboard=True
        ),
    )

```

```
return NOTES
```

```
async def get_notes(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    txt = update.message.text.strip()
    context.user_data["notes"] = "" if txt == t(context, "skip") else txt

    # Підсумок для перевірки
    summary = format_summary(context, for_user=True)
    rows = [[t(context, "confirm_send"), t(context, "restart")]]
    await update.message.reply_text(
        t(context, "summary_intro") + "\n\n" + summary,
        reply_markup=kb(rows),
        parse_mode="HTML",
    )
    return CONFIRM
```

```
async def confirm(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    txt = update.message.text.strip()
    if txt == t(context, "restart"):
        await update.message.reply_text(
            t(context, "restarted"), reply_markup=ReplyKeyboardRemove()
        )
        return await start(update, context)
```

```
# Надсилаємо власнику
owner_msg = format_summary(context, for_user=False)
user = update.effective_user
header = (
    f"🔔 <b>Нова заявка WebKom</b>\n"
    f"Від: <a href='tg://user?id={user.id}'>{user.full_name}</a>"
    f"{' (' + user.username + ') ' if user.username else ''}\n"
    f"Час: {datetime.now().strftime('%d.%m.%Y %H:%M')}\n"
    f"Мова анкети: {context.user_data.get('lang', 'cs').upper()}\n"
    f"_____\n\n"
)
```

```
if OWNER_CHAT_ID:
    try:
        await context.bot.send_message(
            chat_id=OWNER_CHAT_ID,
            text=header + owner_msg,
            parse_mode="HTML",
        )
    except Exception as e:
        logger.error(f"Не вдалося надіслати власнику: {e}")
```

```

        await update.message.reply_text(
            t(context, "thanks"),
            reply_markup=ReplyKeyboardRemove(),
            parse_mode="HTML",
        )
    return ConversationHandler.END

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    await update.message.reply_text(
        t(context, "cancelled") if context.user_data.get("lang") else "OK, cancel",
        reply_markup=ReplyKeyboardRemove(),
    )
    return ConversationHandler.END

# ===== ФОРМАТУВАННЯ ПІДСУМКУ =====
def format_summary(context: ContextTypes.DEFAULT_TYPE, for_user: bool) -> str:
    d = context.user_data
    L = t # shortcut

    lines = [
        f"<b>{L(context, 'sum_name')}</b> {d.get('name', '-')}",
        f"<b>{L(context, 'sum_site_type')}</b> {d.get('site_type', '-')}",
        f"<b>{L(context, 'sum_goal')}</b> {d.get('goal', '-')}",
        f"<b>{L(context, 'sum_pages')}</b> {d.get('pages', '-')}",
        f"<b>{L(context, 'sum_design')}</b> {d.get('design', '-')}",
        f"<b>{L(context, 'sum_content')}</b> {d.get('content', '-')}",
    ]
    if d.get("examples"):
        lines.append(f"<b>{L(context, 'sum_examples')}</b> {d['examples']}")
    lines.append(f"<b>{L(context, 'sum_deadline')}</b> {d.get('deadline', '-')}")
    lines.append(f"<b>{L(context, 'sum_contact')}</b> {d.get('contact', '-')}")
    if d.get("notes"):
        lines.append(f"<b>{L(context, 'sum_notes')}</b> {d['notes']}")
    return "\n".join(lines)

# ===== ЗАПУСК =====
def main() -> None:
    if BOT_TOKEN == "PASTE_YOUR_TOKEN_HERE":
        print("❌ Спочатку встановіть змінну середовища BOT_TOKEN")
        return

    app = Application.builder().token(BOT_TOKEN).build()

```

```

conv = ConversationHandler(
    entry_points=[CommandHandler("start", start)],
    states={
        LANG: [CallbackQueryHandler(set_language, pattern=r"^lang_")],
        NAME: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_name)],
        SITE_TYPE: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_site_
        GOAL: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_goal)],
        PAGES: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_pages)],
        DESIGN: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_design)],
        CONTENT: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_content
        EXAMPLES: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_exempl
        DEADLINE: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_deadli
        CONTACT: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_contact
        NOTES: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_notes)],
        CONFIRM: [MessageHandler(filters.TEXT & ~filters.COMMAND, confirm)],
    },
    fallbacks=[CommandHandler("cancel", cancel)],
    allow_reentry=True,
)

app.add_handler(conv)
logger.info("🤖 WebKom bot crapye...")
app.run_polling(allowed_updates=Update.ALL_TYPES)

if __name__ == "__main__":
    main()

```